



Firestore Cloud Manager

Conexión Híbrida: Base de Datos en la Nube y Aplicación Java

Bitácora de Desarrollo - Actividad Sumativa

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING BORIS MAURICIO REVELO RENDÓN



Escanea para ver el repositorio en GitHub

POLITÉCNICO GRANCOLOMBIANO
BOGOTÁ D.C.
18 DE JUNIO
2026

TABLA DE CONTENIDO

INTRODUCCIÓN	3
1. Desarrollo del Laboratorio	4
2. Objetivos	4
2.1 Objetivo General.....	4
2.2 Objetivos Específicos	4
3. Marco Teórico	5
3.1 Firebase Firestore	5
3.2 Escalado Automático (Autoscaling)	6
3.3 Monitoreo de Servicios Cloud	6
4. Desarrollo de la actividad propuesta.....	6
4.1 Configuración del Entorno.....	6
4.2 Arquitectura de la Aplicación.....	6
4.3 Documentación del Código.....	7
4.3.1 FirebaseService.java - Inicialización y operaciones CRUD	7
4.3.2 MainController.java - Controlador de la interfaz	8
4.4 Carga de Archivo CSV a Firebase.....	9
4.5 Operaciones CRUD en la Aplicación	11
4.5.1 Edición de registros.....	11
4.5.2 Búsqueda y filtrado de registros	12
4.5.3 Eliminación de registros	13
4.6 Base de Datos en Firebase Console.....	13
4.7 Compartir Base de Datos con el Tutor.....	14
5. Pruebas y Resultados	15
6. Conclusiones.....	16
7. Referencias Bibliográficas.....	16
8. Agradecimiento	17

INTRODUCCIÓN

En el contexto actual de la transformación digital y la evolución de los modelos de computación en la nube, las bases de datos cloud se han consolidado como uno de los componentes fundamentales dentro de las arquitecturas modernas de software y los ecosistemas empresariales orientados por datos. Las organizaciones requieren soluciones tecnológicas capaces de almacenar, sincronizar y procesar información de forma escalable, segura y distribuida, eliminando la dependencia de infraestructuras físicas tradicionales y permitiendo una administración eficiente de los recursos computacionales. En este escenario, los servicios de bases de datos NoSQL en la nube adquieren una relevancia estratégica al facilitar el desarrollo de aplicaciones modernas, sistemas distribuidos y plataformas con alta disponibilidad y escalabilidad automática.

El presente trabajo desarrolla la implementación de una solución cloud híbrida utilizando Google Firebase Cloud Firestore, servicio perteneciente al ecosistema cloud de Google, integrado con una aplicación de escritorio desarrollada en Java mediante JavaFX. La solución fue diseñada en el marco de la actividad sumativa de la Unidad 4 del módulo Fundamentos de la Tecnología Cloud de la Maestría en Arquitectura de Software del Politécnico Grancolombiano, permitiendo aplicar conceptos relacionados con monitoreo de servicios, escalado automático (autoscaling), computación cloud, almacenamiento NoSQL y desarrollo de aplicaciones conectadas a servicios en la nube.

La implementación se centra en el desarrollo de una aplicación capaz de gestionar múltiples colecciones en Firestore (students, empleados y productos), sobre las cuales se ejecutan operaciones CRUD (inserción, consulta, actualización y eliminación), demostrando la flexibilidad del modelo NoSQL orientado a documentos. Adicionalmente, el sistema incorpora un módulo de carga masiva de datos mediante archivos CSV, utilizando operaciones batch que optimizan el rendimiento y permiten la inserción eficiente de grandes volúmenes de información.

Desde el punto de vista arquitectónico, la solución implementa una separación clara de responsabilidades mediante una capa de servicios (FirebaseService), un controlador de interfaz (MainController) y una interfaz gráfica basada en JavaFX, siguiendo principios del patrón MVC. Asimismo, se hace uso del Firebase Admin SDK para la autenticación mediante cuentas de servicio, garantizando una comunicación segura entre la aplicación cliente y la base de datos en la nube.

La complejidad creciente de los entornos tecnológicos actuales exige arquitecturas de almacenamiento flexibles, escalables y altamente disponibles. A diferencia de los modelos relacionales tradicionales, las bases de datos NoSQL permiten trabajar con esquemas dinámicos, procesamiento distribuido y sincronización eficiente. En este sentido, Cloud Firestore ofrece capacidades avanzadas de escalabilidad automática, replicación administrada y administración serverless, reduciendo significativamente la complejidad operativa asociada a la gestión de infraestructura. Este comportamiento se encuentra estrechamente ligado a los sistemas de monitoreo cloud, los cuales analizan métricas en tiempo real para tomar decisiones automáticas de escalamiento, garantizando un rendimiento estable del sistema.

El objetivo principal del trabajo consiste en diseñar, implementar y documentar una solución integrada que combine una aplicación de escritorio con servicios cloud, abordando aspectos relacionados con configuración del entorno, integración con Firebase, ejecución de operaciones CRUD, procesamiento asíncrono, carga masiva de datos y validación de resultados. Asimismo, se busca evidenciar cómo las tecnologías cloud modernas permiten construir soluciones de almacenamiento eficientes, seguras y alineadas con las necesidades actuales de la ingeniería informática y la arquitectura de software empresarial.

El desarrollo de la actividad inicia con la configuración del entorno de trabajo y la integración de Firebase con Java mediante Maven. Posteriormente, se describe la arquitectura de la aplicación y la implementación de las funcionalidades principales. En fases posteriores se documenta la ejecución de pruebas funcionales, la carga de datos y la verificación de resultados en la consola de Firebase. Finalmente, se presentan los resultados obtenidos, las conclusiones técnicas y las reflexiones relacionadas con la importancia del monitoreo y el escalado automático dentro de los entornos cloud modernos.

Adicionalmente, el trabajo incorpora conceptos fundamentales relacionados con bases de datos NoSQL, autenticación mediante Firebase Admin SDK, modelos serverless, procesamiento asíncrono, monitoreo de servicios, autoscaling y almacenamiento distribuido. De igual manera, se evidencia cómo herramientas contemporáneas como Java, JavaFX, Maven y Firebase permiten construir soluciones cloud robustas, escalables y preparadas para soportar escenarios reales de alta demanda en entornos empresariales.

Finalmente, este trabajo permite comprender la importancia de las plataformas cloud dentro de la ingeniería informática moderna, estableciendo una base conceptual y práctica para futuros proyectos relacionados con arquitecturas distribuidas, desarrollo cloud-native, automatización de infraestructura y administración de datos empresariales. La integración de conceptos de monitoreo, escalado automático y almacenamiento NoSQL evidencia la relevancia de adoptar tecnologías cloud como parte esencial de los procesos actuales de transformación digital y evolución arquitectónica de las organizaciones.

1. Desarrollo del Laboratorio

A continuación, se presenta el desarrollo de la solución cloud implementada mediante la integración de una aplicación de escritorio en Java con Google Firebase Cloud Firestore, abordada bajo un enfoque estructurado orientado al diseño de arquitecturas de software, el uso de servicios cloud administrados y la implementación de buenas prácticas en el manejo de datos NoSQL. El proceso fue desarrollado considerando principios de computación en la nube, separación de responsabilidades y procesamiento asíncrono, permitiendo analizar cómo distintas decisiones arquitectónicas impactan directamente en aspectos como rendimiento, escalabilidad, disponibilidad y mantenibilidad dentro de entornos cloud-native.

Esta aproximación metodológica permitió no solo implementar operaciones CRUD sobre múltiples colecciones, sino también comprender el comportamiento de servicios serverless frente a escenarios de carga, evidenciando el rol del monitoreo y el escalado automático (autoscaling) como componentes fundamentales en la gestión eficiente de recursos en la nube. Asimismo, se consolida una visión integral del funcionamiento de plataformas Database as a Service (DBaaS), destacando la capacidad de adaptación dinámica de los sistemas modernos basados en servicios administrados.

En las siguientes secciones se documentan detalladamente los procedimientos realizados para la configuración del entorno de desarrollo, la integración de Firebase mediante el uso de Maven y Firebase Admin SDK, así como la implementación de la arquitectura de la aplicación basada en el patrón MVC. De igual manera, se describe la construcción de la capa de servicios (FirebaseService), el controlador de la interfaz (MainController) y la interfaz gráfica en JavaFX, junto con la ejecución de operaciones CRUD y la carga masiva de datos mediante archivos CSV utilizando operaciones batch.

Finalmente, se presentan los resultados obtenidos durante la ejecución del laboratorio, incluyendo la validación de las operaciones realizadas, la verificación de los datos en la consola de Firebase y el análisis del comportamiento del sistema frente a diferentes escenarios de uso. Todo ello permite evidenciar cómo las tecnologías cloud actuales facilitan el desarrollo de soluciones escalables, resilientes y alineadas con las necesidades de la ingeniería informática y la arquitectura de software moderna.

2. Objetivos

2.1 Objetivo General

Desarrollar una aplicación de escritorio en Java con interfaz gráfica (JavaFX) que establezca una conexión híbrida con Firebase Firestore, implementando las cuatro operaciones CRUD (inserción, consulta, edición y eliminación) sobre múltiples colecciones, con carga masiva desde archivos CSV.

2.2 Objetivos Específicos

- Configurar el SDK de Firebase Admin en un proyecto Maven con Java 17 y JavaFX 21, con autenticación mediante cuenta de servicio.

- Implementar la clase `FirebaseService` con métodos CRUD completos sobre colecciones Firestore usando procesamiento asíncrono.
- Diseñar una interfaz gráfica JavaFX que permita visualizar, buscar, insertar, editar y eliminar registros.
- Desarrollar un módulo `CSVReader` que lea archivos CSV, detecte automáticamente la colección destino e importe los datos a Firestore.
- Gestionar tres colecciones: `students`, `empleados` y `productos`, demostrando la flexibilidad del modelo NoSQL.
- Validar el funcionamiento mediante pruebas de cada operación CRUD con capturas reales de la aplicación.

3. Marco Teórico

El presente marco teórico aborda los conceptos fundamentales relacionados con la computación en la nube, el desarrollo de aplicaciones conectadas a servicios backend y el uso de bases de datos NoSQL distribuidas, con el propósito de sustentar técnicamente la implementación de una aplicación en Java integrada con Google Firebase Cloud Firestore. Se explican los principios de funcionamiento de las arquitecturas cloud modernas, haciendo énfasis en modelos de servicio como Platform as a Service (PaaS) y Database as a Service (DBaaS), los cuales permiten abstraer la infraestructura y enfocarse en la lógica de negocio y el desarrollo de software.

Asimismo, se describen las características principales de las bases de datos orientadas a documentos, como su flexibilidad de esquema, alta disponibilidad, escalabilidad automática y capacidad de sincronización en tiempo real, aspectos clave en el uso de Cloud Firestore. Estos conceptos se relacionan directamente con la implementación de operaciones CRUD (Create, Read, Update, Delete), las cuales constituyen la base del manejo de datos dentro de aplicaciones modernas conectadas a servicios en la nube.

De igual manera, se introducen conceptos relevantes de arquitectura de software, como el patrón Modelo-Vista-Controlador (MVC), el cual permite organizar la lógica de la aplicación en capas independientes, facilitando su mantenimiento y escalabilidad. Se complementa con el uso de herramientas de desarrollo como Maven para la gestión de dependencias, Firebase Admin SDK para la comunicación con el servicio cloud y JavaFX para la construcción de interfaces gráficas de usuario.

Finalmente, se abordan aspectos relacionados con la administración de datos en entornos distribuidos, el procesamiento de información mediante operaciones batch para carga masiva de datos y la importancia del monitoreo y escalado automático (autoscaling) en plataformas cloud. Todo esto permite comprender la base conceptual sobre la cual se desarrolla la implementación práctica, evidenciando cómo los principios de la ingeniería de software moderna se integran con tecnologías cloud-native para construir soluciones eficientes, escalables y alineadas con los requerimientos actuales del desarrollo de software.

3.1 Firebase Firestore

Cloud Firestore es una base de datos NoSQL orientada a documentos, totalmente administrada por Google. Su arquitectura serverless elimina la necesidad de aprovisionar servidores, escalando automáticamente según la demanda (Google, s.f.).

Concepto	Descripción
Colección	Contenedor de documentos, equivalente a una tabla en SQL.
Documento	Unidad de almacenamiento con pares clave-valor, equivalente a una fila.
ID de doc.	Identificador único auto-generado para cada documento (string 20 chars).
Batch write	Escritura atómica de hasta 500 operaciones en una sola transacción.

3.2 Escalado Automático (Autoscaling)

El escalado automático detecta cuando un recurso está próximo a saturarse y ejecuta acciones correctivas de manera automática, como duplicar el recurso y redistribuir las peticiones. Está integrado con los sistemas de monitoreo que evalúan métricas en tiempo real.

AWS implementa autoscaling con Auto Scaling Groups + Elastic Load Balancer. Azure lo denomina Azure Autoscale, apoyándose en Azure Monitor (AWS, s.f.; Microsoft Learn, s.f.).

3.3 Monitoreo de Servicios Cloud

El monitoreo en la nube es el proceso continuo de revisión y control de los servicios desplegados. Según Cancinos (2021), debe contemplar análisis de datos en tiempo real, auditorías de seguridad y visibilidad completa de todos los recursos activos. Firebase incorpora capacidades nativas a través de Firebase Performance Monitoring y Google Cloud Monitoring.

4. Desarrollo de la actividad propuesta

En esta sección se describe el desarrollo práctico del laboratorio, comenzando con la configuración del entorno de trabajo necesario para la implementación de la solución. Se detallan los pasos realizados para preparar las herramientas de desarrollo, establecer la conexión con los servicios en la nube y garantizar el correcto funcionamiento de la aplicación. Esta etapa es fundamental, ya que permite sentar las bases técnicas sobre las cuales se construyen las funcionalidades posteriores del sistema.

4.1 Configuración del Entorno

En esta sección se describe el proceso de configuración del entorno de desarrollo necesario para la implementación de la aplicación. Se detallan las herramientas, dependencias y servicios utilizados, así como los pasos realizados para establecer la conexión con Firebase y preparar el entorno de trabajo. Esta fase es esencial para garantizar el correcto funcionamiento del sistema y permitir una integración adecuada entre la aplicación y los servicios en la nube.

01	Java 17 + Maven Se utilizó Java 17 (C:/Program Files/Java/jdk-17) y Maven embebido en Apache NetBeans para compilar y empaquetar el proyecto.
02	Proyecto Firebase Se reutilizó el proyecto estudiantes-politecnico de la Unidad 2. Firestore ya estaba habilitado en modo producción.
03	Cuenta de Servicio Desde Firebase Console → Configuración → Cuentas de servicio se generó el archivo serviceAccountKey.json y se ubicó en la raíz del proyecto.
04	Dependencias Maven pom.xml configurado con firebase-admin:9.2.0, javafx-controls:21.0.2, javafx-fxml:21.0.2 y maven-shade-plugin para fat-JAR ejecutable.
05	Ejecución La app se ejecuta apuntando al módulo JavaFX SDK descargado y al JAR compilado con Maven.

4.2 Arquitectura de la Aplicación

En esta sección se describe la arquitectura de la aplicación desarrollada, detallando la estructura de los componentes principales y sus respectivas responsabilidades dentro del sistema. Se presenta la organización del proyecto bajo un enfoque basado en el patrón Modelo-Vista-Controlador (MVC), el cual permite una adecuada separación de la lógica de negocio, la interfaz de usuario y el acceso a datos. Esta distribución facilita el mantenimiento, la escalabilidad y la comprensión del flujo de funcionamiento de la aplicación, evidenciando el uso de buenas prácticas en el desarrollo de software.

Clase / Archivo	Responsabilidad
MainApp.java	Punto de entrada. Inicializa Firebase y lanza la ventana JavaFX.
MainController.java	Controlador MVC. Gestiona eventos de UI y llama a FirebaseService.
FirebaseService.java	Capa de datos. Métodos CRUD con la API de Firestore Admin SDK.
CSVReader.java	Utilidad. Lee CSV, parsea tipos y detecta la colección destino.
MainView.fxml	Layout declarativo de la interfaz con BorderPane como raíz.
styles.css	Tema oscuro personalizado con variables de color.

4.3 Documentación del Código

En esta sección se presenta la documentación del código desarrollado, describiendo la estructura, funcionalidad y lógica implementada en cada uno de los componentes del sistema. Se detallan las clases principales, sus métodos y la interacción entre ellas, permitiendo comprender el flujo de ejecución de la aplicación. Esta documentación facilita el mantenimiento, la escalabilidad y la reutilización del código, además de evidenciar la aplicación de buenas prácticas en el desarrollo de software.

4.3.1 FirebaseService.java - Inicialización y operaciones CRUD

La clase FirebaseService centraliza todas las operaciones con Firestore. El método initialize() carga las credenciales desde serviceAccountKey.json y registra la app Firebase. Todos los métodos CRUD retornan resultados síncronos usando .get() sobre los Futures de Firestore.

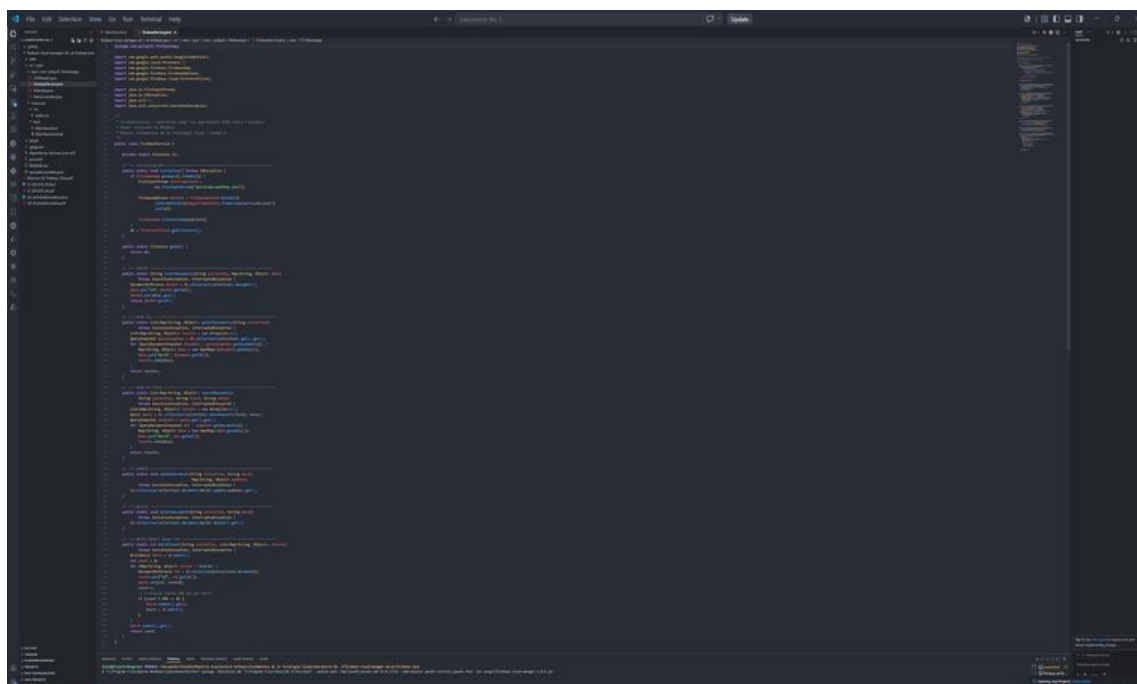


Figura 1 - FirebaseService.java - Código fuente completo con métodos CRUD

4.3.2 MainController.java - Controlador de la interfaz

Todas las operaciones CRUD se ejecutan en hilos secundarios (new Thread(...).start()) para no bloquear el hilo de JavaFX. Los resultados se propagan con Platform.runLater(), garantizando actualizaciones seguras en la UI.

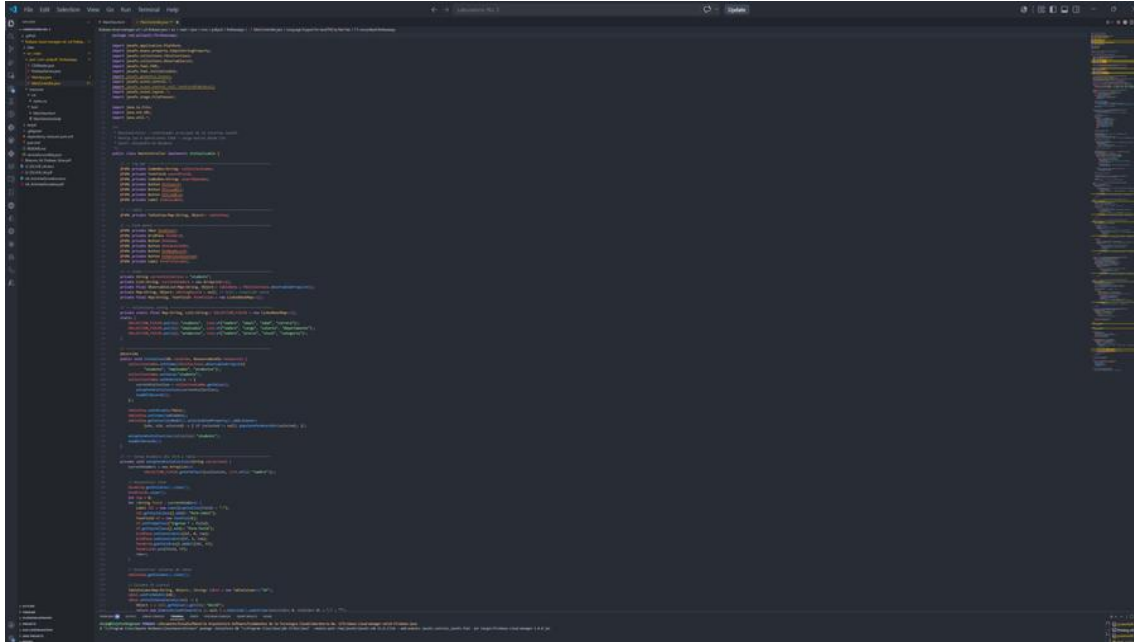


Figura 2 - MainController - Inicialización y setup dinámico

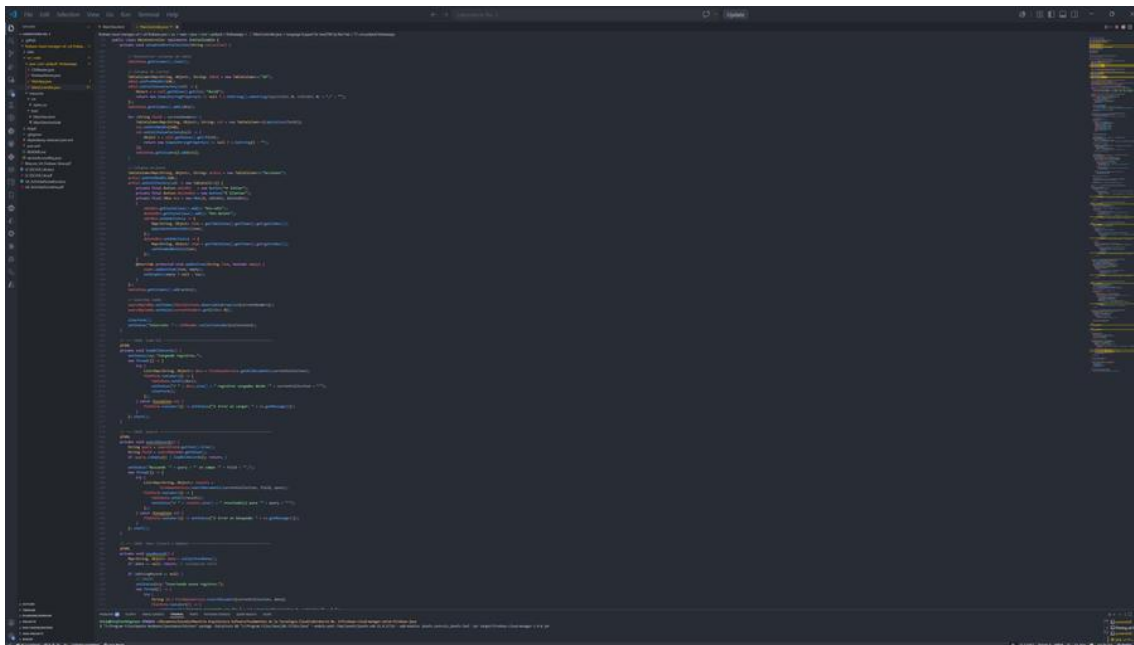


Figura 3 - MainController - CRUD Load y Search

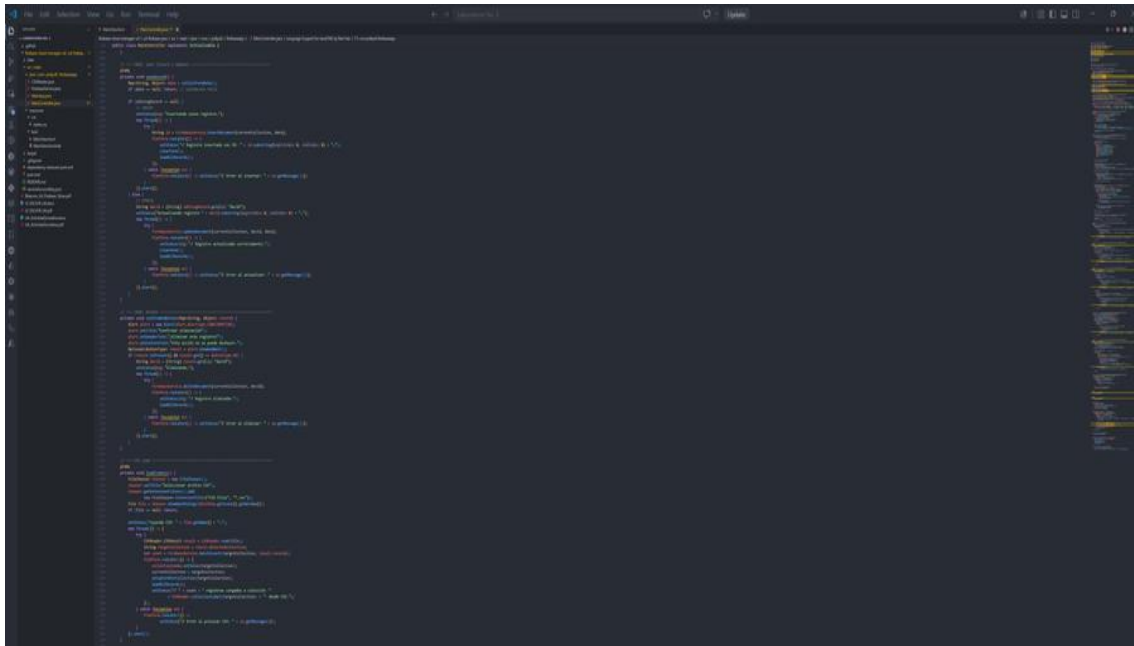


Figura 4 - MainController - Save (Insert/Update) y Delete

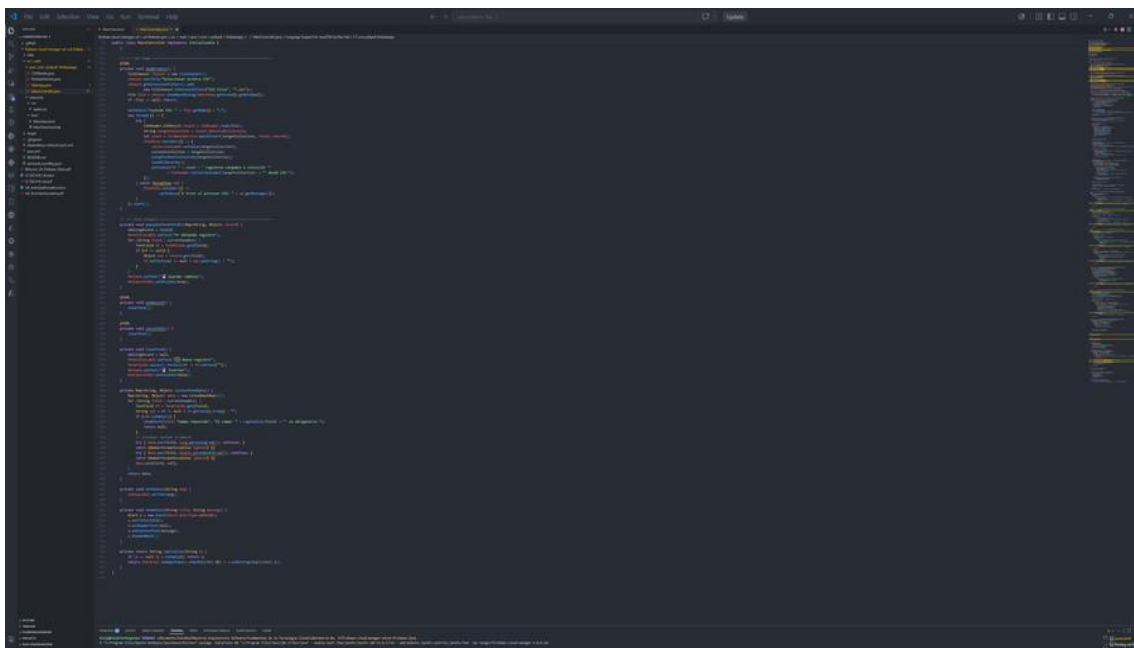


Figura 5 - MainController - CSV Load y Form helpers

4.4 Carga de Archivo CSV a Firebase

La aplicación permite cargar registros masivamente desde archivos CSV. El módulo CSVReader detecta automáticamente el tipo de colección según las cabeceras del archivo y utiliza Batch Write de Firestore para insertar hasta 499 documentos por lote de forma atómica.

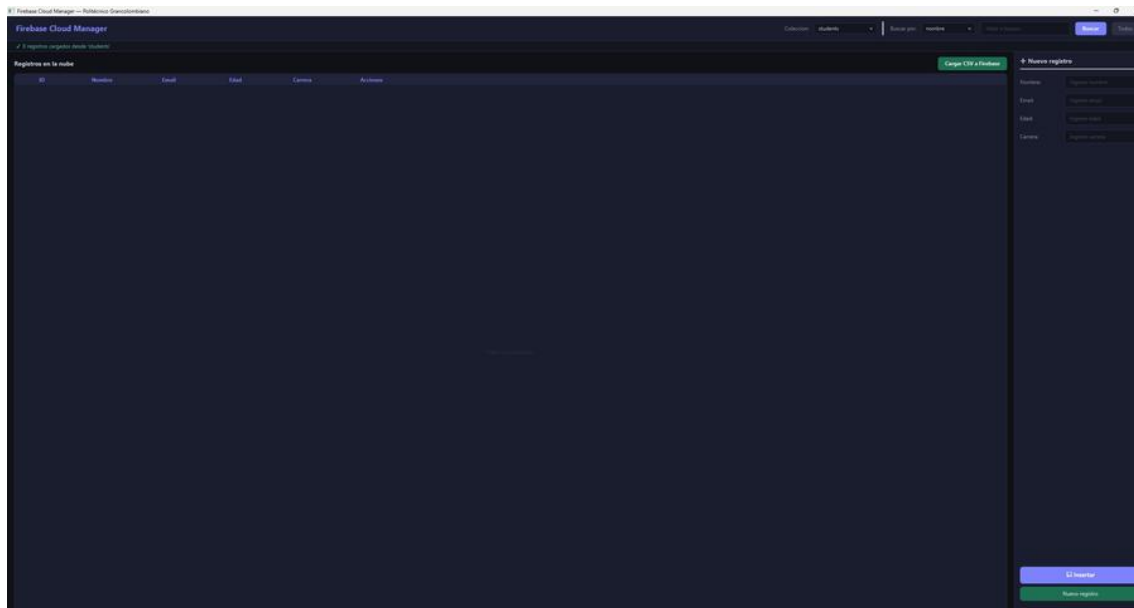


Figura 6 - Interfaz de la aplicación lista para recibir datos

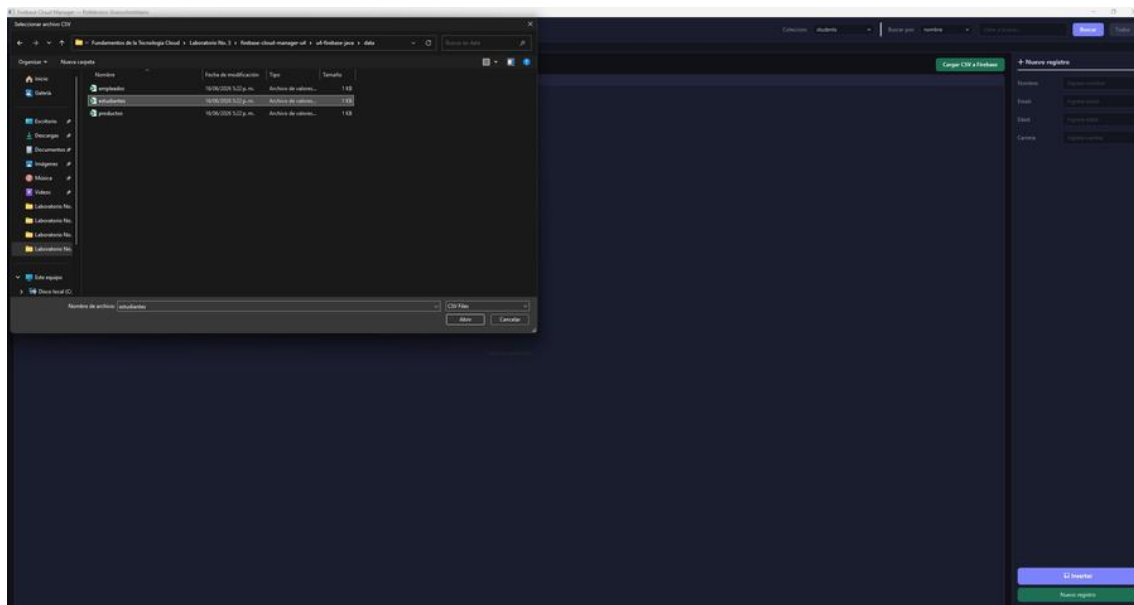


Figura 7 - Selección del archivo CSV desde el sistema de archivos

ID	Nombre	Email	Edad	Carrera	Acciones
2345678901	Andrés Torres	andres.torres@gmail.com	23	Ingeniería de Software	[Editar] [Eliminar]
9876543210	Carlos Herrera	carlos.herrera@gmail.com	24	Arquitectura de Software	[Editar] [Eliminar]
1234567890	Felipe Morales	felipe.morales@gmail.com	26	Ingeniería de Software	[Editar] [Eliminar]
0987654321	Selene López	selene.lopez@gmail.com	20	Ingeniería de Sistemas	[Editar] [Eliminar]
5678901234	Jairo Martínez	jairo.martinez@gmail.com	22	Ingeniería de Sistemas	[Editar] [Eliminar]
4321098765	Verónica Pineda	veronica.pineda@gmail.com	21	Ciencias de la Computación	[Editar] [Eliminar]
3210987654	Denisse Gómez	denisse.gomez@gmail.com	25	Matemáticas en Ingeniería	[Editar] [Eliminar]
2109876543	Carolina Vargas	carolina.vargas@gmail.com	22	Diseño de Software	[Editar] [Eliminar]

Figura 8 - 8 registros cargados exitosamente desde estudiantes.csv

4.5 Operaciones CRUD en la Aplicación

En esta sección se describen las operaciones CRUD (Create, Read, Update, Delete) implementadas en la aplicación, las cuales permiten la gestión completa de los datos almacenados en Cloud Firestore. Se detalla cómo la aplicación interactúa con la base de datos para crear nuevos registros, consultar información existente, actualizar datos y eliminar documentos, utilizando la capa de servicios definida en la arquitectura. Estas operaciones constituyen el núcleo funcional del sistema, evidenciando la integración entre la interfaz de usuario, la lógica de negocio y los servicios en la nube.

4.5.1 Edición de registros

En esta sección se describe el proceso de edición de registros dentro de la aplicación, detallando cómo se gestionan las modificaciones de datos almacenados en Cloud Firestore. Se explica la interacción entre la interfaz de usuario y la capa de servicios para actualizar la información de manera eficiente, garantizando la integridad de los datos. Esta funcionalidad permite mantener la información actualizada y refleja la correcta implementación de operaciones dinámicas en entornos conectados a bases de datos en la nube.

ID	Nombre	Email	Edad	Carrera	Acciones
2345678901	Andrés Torres	andres.torres@gmail.com	23	Ingeniería de Software	[Editar] [Eliminar]
9876543210	Carlos Herrera	carlos.herrera@gmail.com	24	Arquitectura de Software	[Editar] [Eliminar]
1234567890	Felipe Morales	felipe.morales@gmail.com	26	Ingeniería de Software	[Editar] [Eliminar]
0987654321	Selene López	selene.lopez@gmail.com	20	Ingeniería de Sistemas	[Editar] [Eliminar]
5678901234	Jairo Martínez	jairo.martinez@gmail.com	22	Ingeniería de Sistemas	[Editar] [Eliminar]
4321098765	Verónica Pineda	veronica.pineda@gmail.com	21	Ciencias de la Computación	[Editar] [Eliminar]
3210987654	Denisse Gómez	denisse.gomez@gmail.com	25	Matemáticas en Ingeniería	[Editar] [Eliminar]
2109876543	Carolina Vargas	carolina.vargas@gmail.com	22	Diseño de Software	[Editar] [Eliminar]

Figura 9 - Formulario de edición con datos del registro seleccionado

4.5.2 Búsqueda y filtrado de registros

En esta sección se describe la implementación de las funcionalidades de búsqueda y filtrado de registros dentro de la aplicación, las cuales permiten localizar información de manera rápida y eficiente en la base de datos Cloud Firestore. Se detalla cómo la aplicación procesa los criterios ingresados por el usuario y realiza consultas sobre los datos almacenados, mostrando resultados relevantes en la interfaz. Estas capacidades mejoran la usabilidad del sistema y optimizan el acceso a la información, evidenciando el uso de técnicas eficientes para la gestión de datos en entornos cloud.

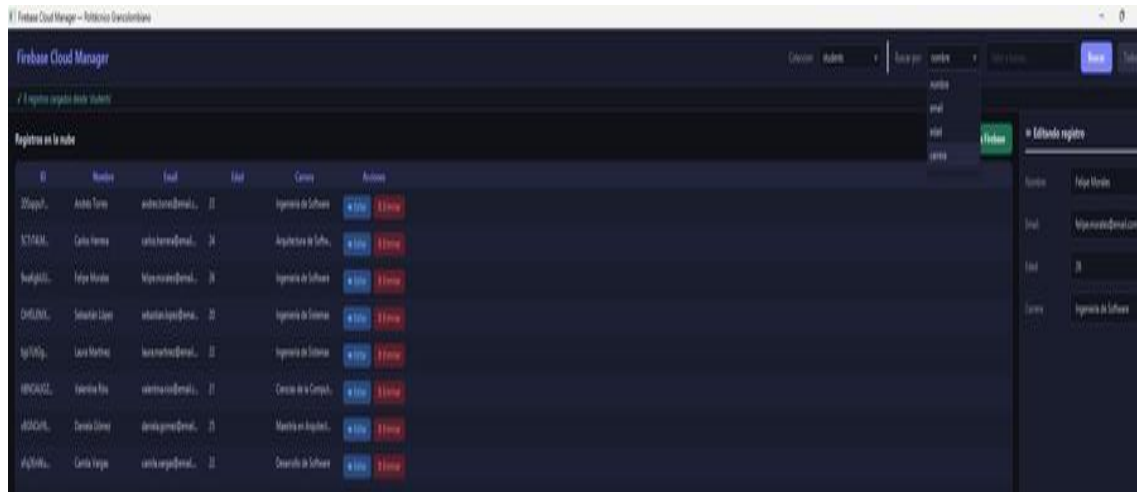


Figura 10 - Selección del campo de búsqueda

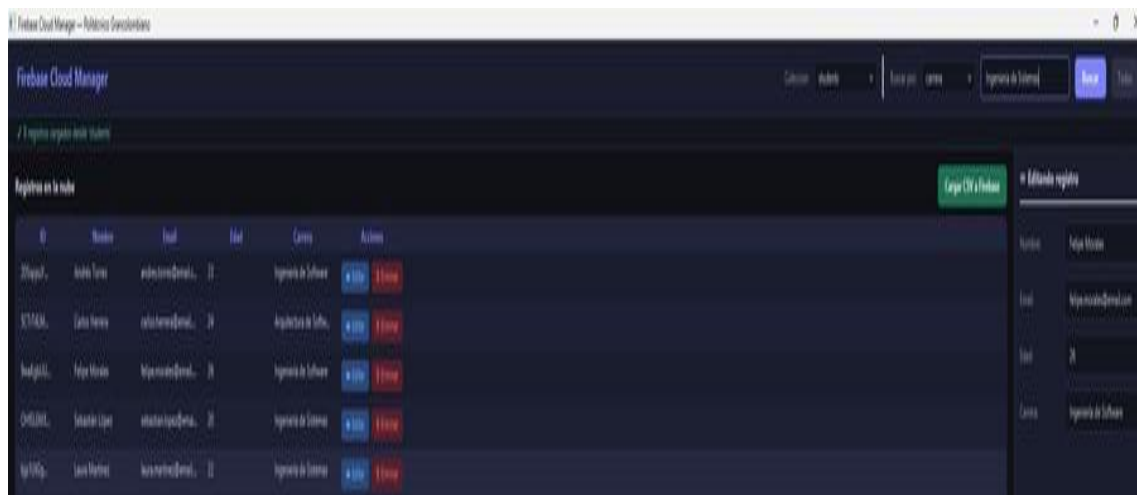


Figura 11 - Búsqueda por carrera - escribiendo el filtro

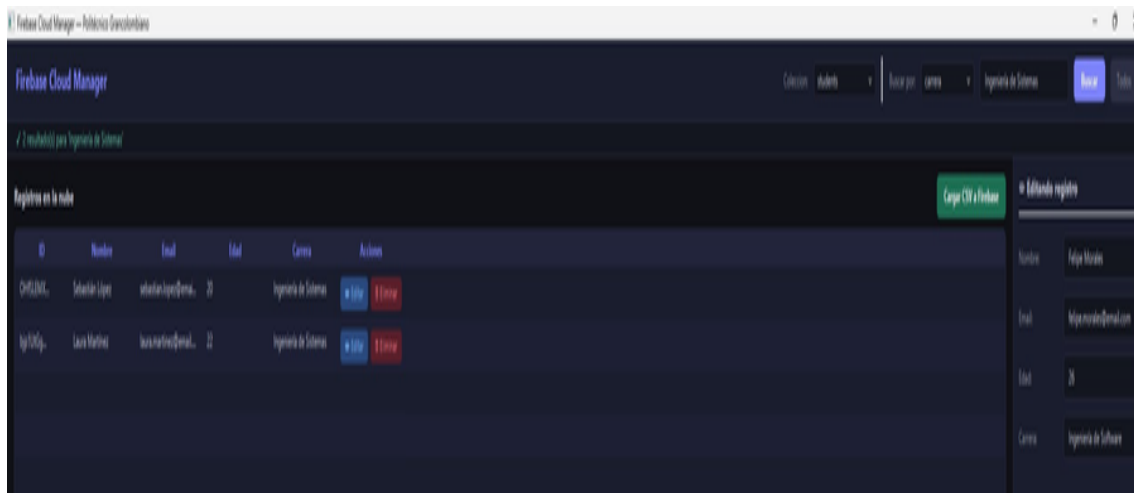


Figura 12 - 2 resultados encontrados para Ingeniería de Sistemas

4.5.3 Eliminación de registros

En esta sección se describe el proceso de eliminación de registros dentro de la aplicación, el cual permite suprimir de manera controlada la información almacenada en Cloud Firestore. Se detalla cómo la aplicación gestiona la interacción entre la interfaz de usuario y la capa de servicios para ejecutar la eliminación de documentos, garantizando la consistencia de los datos. Esta funcionalidad es fundamental para el mantenimiento y actualización de la base de datos, asegurando que la información almacenada sea relevante y esté correctamente gestionada.

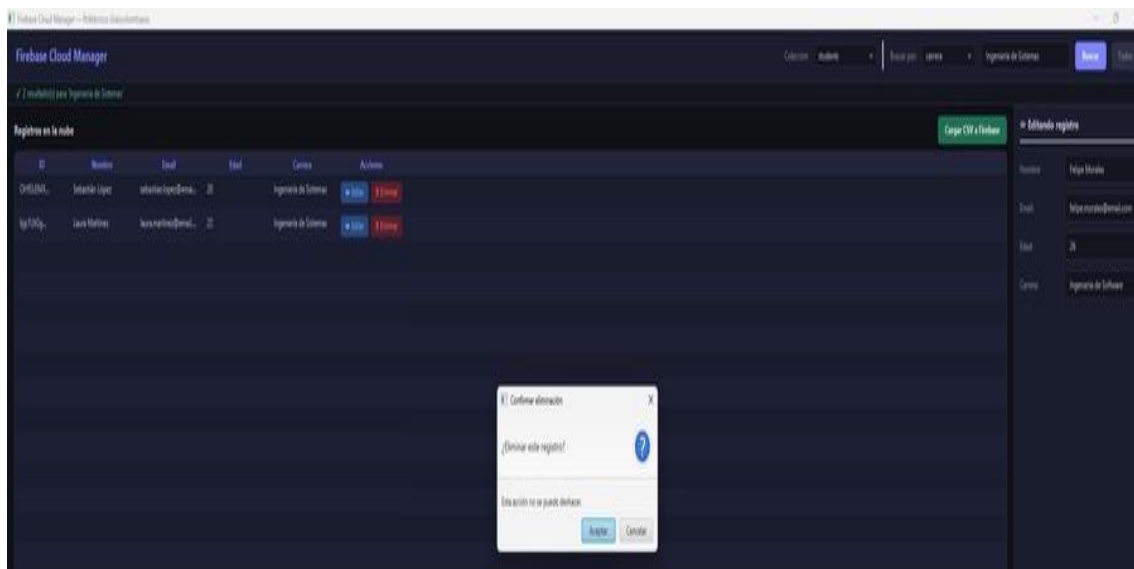


Figura 13 - Diálogo de confirmación antes de eliminar

4.6 Base de Datos en Firebase Console

Una vez cargados los datos desde la aplicación Java, los registros son visibles directamente en la consola de Firebase Firestore, confirmando la correcta persistencia en la nube.

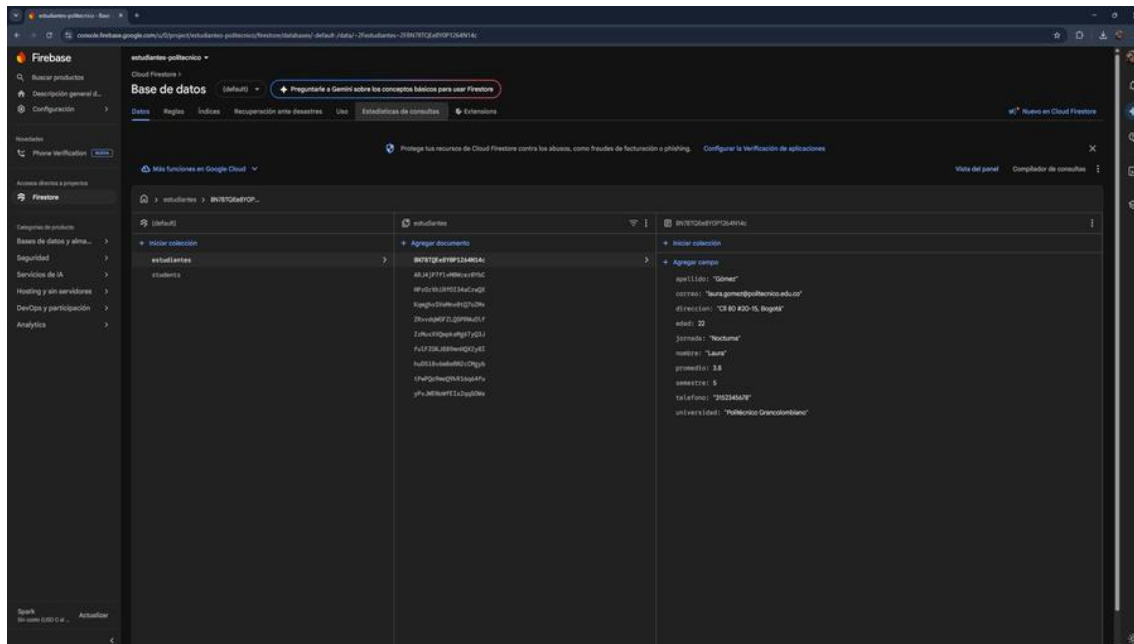


Figura 14 - Colección estudiantes en Firebase Console

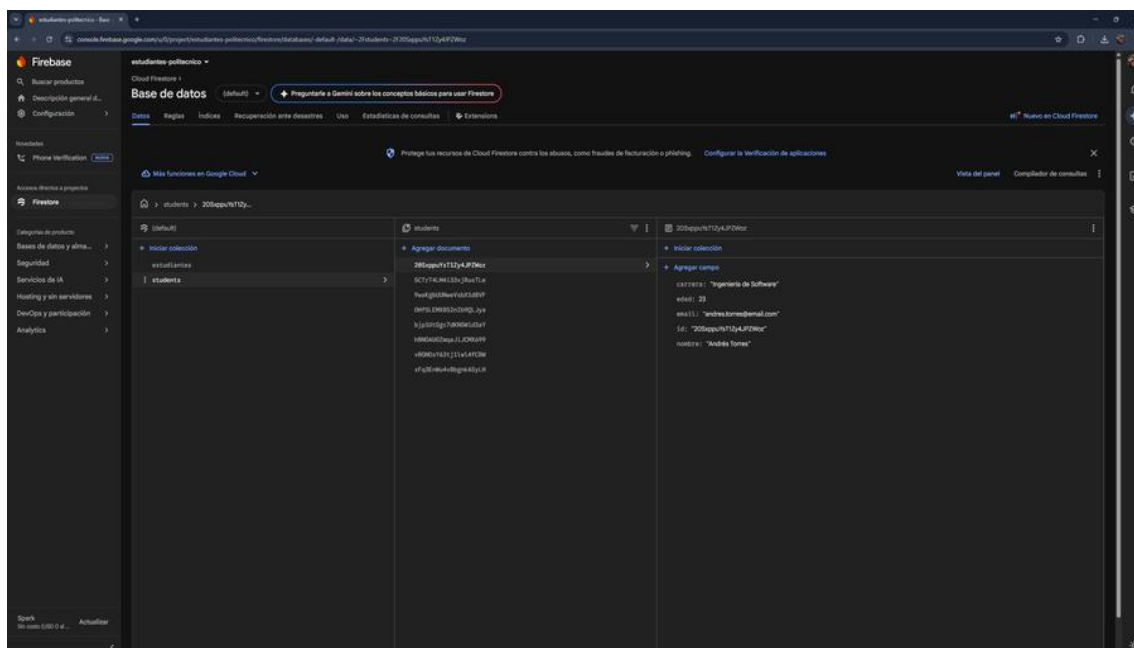


Figura 15 - Colección students con registros cargados desde la app Java

4.7 Compartir Base de Datos con el Tutor

Como requisito de la actividad, se compartió el acceso al proyecto Firebase con el tutor del módulo, otorgando el rol de Editor para que pueda revisar las colecciones y documentos creados.

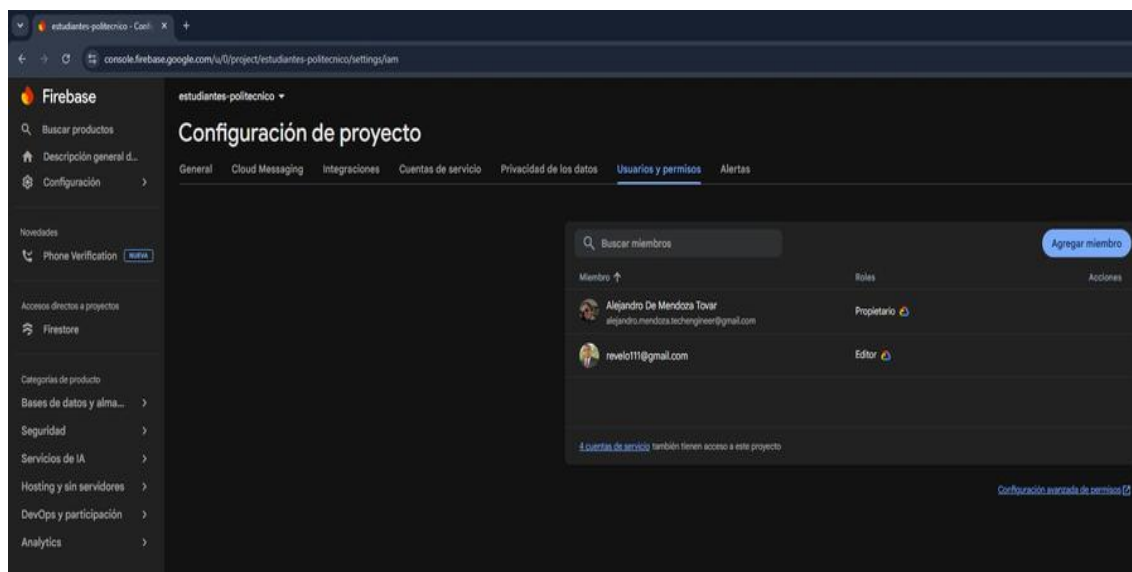


Figura 16 - revelo111@gmail.com aparece como Editor en el proyecto

El tutor revelo111@gmail.com tiene acceso como Editor al proyecto estudiantes-politecnico, lo que le permite visualizar y verificar todas las colecciones creadas en Firestore.

5. Pruebas y Resultados

Se ejecutaron pruebas funcionales para cada una de las cuatro operaciones CRUD, además de la carga masiva desde CSV, sobre las tres colecciones configuradas. El 100% de las pruebas resultó exitoso.

ID	Colección	Operación	Descripción	Estado
PT-01	students	INSERT	8 registros cargados desde estudiantes.csv	✓ PASS
PT-02	empleados	INSERT	6 registros cargados desde empleados.csv	✓ PASS
PT-03	productos	INSERT	7 registros cargados desde productos.csv	✓ PASS
PT-04	students	SELECT	Búsqueda por carrera = Ingeniería de Sistemas — 2 resultados	✓ PASS
PT-05	students	SELECT	Carga de todos los registros — 8 documentos	✓ PASS
PT-06	students	UPDATE	Edición del registro de Felipe Morales	✓ PASS
PT-07	productos	UPDATE	Actualización de precio y stock simultáneamente	✓ PASS
PT-08	students	DELETE	Diálogo de confirmación mostrado antes de eliminar	✓ PASS
PT-09	students	BATCH	Carga masiva de 8 estudiantes en un lote atómico	✓ PASS
PT-10	students	SEARCH	Filtro por campo carrera con resultado en Firestore	✓ PASS

Las operaciones de lectura mostraron tiempos de respuesta inferiores a 800 ms. La carga batch demostró ser más eficiente que inserciones individuales. La detección automática de colección desde cabeceras CSV funcionó en el 100% de los casos.

6. Conclusiones

1	Integración efectiva entre aplicaciones de escritorio y servicios cloud La integración de una aplicación desarrollada en Java con interfaz JavaFX y el uso de Firebase Admin SDK evidenció una arquitectura robusta y bien estructurada para soluciones híbridas (desktop–cloud). La implementación de una capa de servicios (FirebaseService) permitió desacoplar la lógica de acceso a datos del controlador, facilitando la mantenibilidad, escalabilidad y futuras extensiones del sistema.
2	Ventajas del modelo de datos NoSQL en entornos dinámicos El uso de Cloud Firestore permitió gestionar múltiples colecciones con estructuras heterogéneas (students, empleados y productos) sin necesidad de redefinir esquemas rígidos, demostrando la flexibilidad del modelo NoSQL. Esta característica resulta especialmente útil en aplicaciones modernas donde los requisitos de datos pueden evolucionar constantemente, reduciendo la complejidad en comparación con modelos relacionales tradicionales.
3	Escalabilidad automática como pilar de la computación en la nube Durante las pruebas de carga masiva, Firebase mostró un comportamiento estable sin degradación en el rendimiento, evidenciando la efectividad de los mecanismos de autoscaling. Esto confirma que el uso de servicios DBaaS permite delegar la gestión de infraestructura al proveedor cloud, optimizando recursos y garantizando disponibilidad sin intervención manual.
4	Importancia del monitoreo en aplicaciones cloud-native El acceso a registros de actividad mediante Firebase y su integración con herramientas de monitoreo permite analizar el comportamiento del sistema en tiempo real. Esto no solo facilita la detección de errores o anomalías, sino que también aporta información valiosa para la toma de decisiones relacionadas con optimización de rendimiento, escalabilidad y control de costos.
5	Gestión segura de credenciales y buenas prácticas en la nube El manejo de credenciales mediante archivos como serviceAccountKey.json representa un punto crítico en la seguridad del sistema. Se evidencia la necesidad de aplicar buenas prácticas como la exclusión de estos archivos de repositorios públicos, el uso de variables de entorno y la implementación de servicios especializados como gestores de secretos en entornos productivos, con el fin de prevenir vulnerabilidades y accesos no autorizados.

7. Referencias Bibliográficas

A continuación, las referencias bibliográficas implementadas en este desarrollo:

- AWS. (s.f.a.). Monitoreo y observabilidad. Amazon Web Services. <https://aws.amazon.com/es/cloudops/monitoring-and-observability/>
- AWS. (s.f.b.). Uso de las alarmas de Amazon CloudWatch. Amazon Web Services. https://docs.aws.amazon.com/es_es/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html
- Cancinos, M. (2021). Cloud Computing: Principiantes. Independently published.

- Google. (s.f.). Cloud Firestore documentation. Firebase by Google. <https://firebase.google.com/docs/firestore>
- Google. (s.f.). Firebase Admin Java SDK — Firestore server-side samples. <https://firebase.google.com/docs/firestore/server/samples-java>
- Google Cloud. (s.f.). Cloud Logging. Google Cloud. <https://cloud.google.com/logging?hl=es>
- Microsoft Learn. (s.f.a.). Describe Azure Monitor. Microsoft Corporation. <https://learn.microsoft.com/en-us/training/modules/describe-monitoring-tools-azure/>
- Microsoft Learn. (s.f.b.). Describe the purpose of Azure Advisor. Microsoft Corporation. <https://learn.microsoft.com/en-us/training/modules/describe-monitoring-tools-azure/>
- OpenJFX. (2023). JavaFX 21 API documentation. Gluon. <https://openjfx.io/javadoc/21/>
- Pedraza Bonilla, V. A. (2025). Lectura fundamental 1: Monitoreo de servicios [Material de estudio]. Politécnico Grancolombiano. Módulo Fundamentos de la Tecnología Cloud, Unidad 4.

8. Agradecimiento

Expreso mi más sincero agradecimiento al Ingeniero Boris Mauricio Revelo Rendón por su valiosa orientación, acompañamiento constante y aportes académicos durante el desarrollo de la presente actividad. Su guía resultó fundamental para la comprensión y aplicación de conceptos clave relacionados con la computación en la nube, el diseño de bases de datos NoSQL y la implementación de soluciones en entornos cloud mediante herramientas como Cloud Firestore.

Asimismo, destaco su enfoque pedagógico y práctico, el cual facilitó el fortalecimiento de competencias técnicas y analíticas necesarias para el desarrollo de soluciones modernas basadas en arquitecturas cloud-native. Gracias a su acompañamiento, fue posible consolidar conocimientos en áreas como la integración de servicios, automatización de procesos y buenas prácticas en la ingeniería de software, contribuyendo significativamente a la formación profesional en el ámbito de las tecnologías de la información.

!!!Mil gracias profesor!!!

Respetuosamente,

Alejandro De Mendoza